

Chessboard detection

Diogo Sarmento
Faculty of Engineering of University
of Porto
Porto, Portugal
up202109663@up.pt

Tomás Sarmento
Faculty of Engineering of University
of Porto
Porto, Portugal
up202108778@up.pt

Rodrigo Póvoa
Faculty of Engineering of University
of Porto
Porto, Portugal
up202108890@up.pt

1 TASK 2

1.1 Introduction

The primary objective of Task 2 is to develop a Convolutional Neural Network (CNN) based model to accurately determine the total number of chess pieces present on a chessboard in a given image. The number of pieces can range from a minimum of 2 to a maximum of 32. This problem was approached as a regression task, aiming to predict a continuous value representing the piece count.

To find the most effective solution, several modeling strategies and architectures were explored. This report details the methodology, from dataset preparation and data augmentation to the various modeling techniques implemented, and presents a summary of the results achieved.

1.2 Methodology

The project utilizes the Chess Pieces Dataset, with predefined training, validation, and test splits as specified in the annotations.json file. The dataset contains a diverse set of images of chess games in various stages for this task we are able to use the full 10.000 images because we don't need the bounding boxes of the pieces.

Given the large size of the images, a crucial pre-processing step involving data processing, knowing that we are using resnet pretrained models we have to pre-process the images with a few transformations, which is the standard input size for many pretrained CNN architectures. This resizing helps in reducing computation time and memory usage during training.

Two primary data transformation pipelines were established:

- data_in: A standard pipeline for validation and testing, which includes resizing, center cropping, and normalization using ImageNet statistics.
- data_aug: A more extensive augmentation pipeline for training. This pipeline was designed to improve model generalization and prevent overfitting by introducing variations into the training data. The augmentations included:
 - (1) Random Resized Crop
 - (2) Random Horizontal Flip
 - (3) Color Jitter (adjusting brightness, contrast, saturation, and hue)
 - (4) Random Rotation

1.3 Modeling Approaches

Initially, the problem was modeled as a multiclass and a multi-label classification task.

- Multiclass: Each possible piece count (from 2 to 32) was treated as a separate class.
- Multilabel: Each of the 64 squares on the board was treated as a binary classification target (piece present or not).

The most successful approach was to treat the problem as a regression task, where the model directly predicts the number of pieces.

- Architecture: We leveraged the power of transfer learning by using pretrained models. The primary models tested were ResNet50 and the more complex ResNeXt101_32x8D. The final classification layer of these models was replaced with a single output neuron to produce a continuous value.
- Output Activation: To constrain the model's output to the valid range of piece counts [0, 32], a Sigmoid activation function was applied to the final output, which was then scaled by 32. This ensures the prediction remains within a logical bound.
- Loss Function and Metric: The model was trained using a regression-appropriate loss function, such as Mean Squared Error (MSE) or L1 Loss (Mean Absolute Error). The primary metric for evaluating performance across all experiments was the Mean Absolute Error (MAE), which provides a clear and interpretable measure of the average error in the predicted piece count.

1.4 Results

The various modeling strategies were evaluated based on their Mean Absolute Error on the validation and test sets. The regression approach consistently outperformed the classification baselines.

The best results were achieved using the ResNeXt101_32x8D architecture combined with the extensive data augmentation pipeline (data_aug). This configuration demonstrated strong performance and generalization.

The final results are summarized below:

Table 1: Comparison of different models and approaches on Mean Absolute Error (MAE).

Model	Approach	Data Aug.	Test MAE
ResNet50	Multilabel	No	2.490
ResNet50	Sigmoid Regression	No	1.181
ResNet50	Sigmoid Regression	Extensive	0.763
ResNeXt101	Sigmoid Regression	Extensive	0.671

The model achieved a Mean Absolute Error of 0.671 on the test set, indicating that, on average, the model's prediction is less than one piece away from the actual count. This level of accuracy is highly effective for the given task.

1.5 Conclusion

For the task of counting chess pieces, a regression-based approach using a deep CNN proved to be highly successful. The combination of the pretrained ResNeXt101_32x8D model, extensive data augmentation, and a scaled Sigmoid activation function provided the best performance. The final model is capable of predicting the number of pieces on the board with a high degree of accuracy, achieving a Mean Absolute Error of just 0.671 on the unseen test data.

2 TASK 3

2.1 Introduction

The task 3 is split into two different tasks:

- task 3.1 : that focuses on accurately determining the bounding boxes of the pieces presented on the chessboard.
- task 3.2 : that focuses on accurately creating a digital twin of the chessboard depicted on the image.

Several approaches were implemented and will be explored in this report.

3 TASK 3.1

3.1 Introduction

The Task 3.1 of this project focuses on the detection and classification of chess pieces from a single image. The primary goal is to accurately identify the location of each piece on the board by predicting its bounding box and correctly classifying its type and color. This task serves as the base for Task 3.2, which involves creating a full "digital twin" of the chessboard.

3.2 Methodology

We are utilizing the Chess Detection Dataset from the 4TU data repository. As specified in the project guidelines, we used the chessred2k data split defined in the annotations.json file. The initial annotations were provided in a COCO-like format. To make them compatible with the YOLO architecture, we used a Python script (ParseToYoloFormat.py) to convert the annotations. This script performs the following transformation for each piece's bounding box:

- (1) Reads the original bounding box, which is in the format [x_min, y_min, width, height].
- (2) Converts it to the YOLO format: [class_id] [x_center_norm] [y_center_norm] [width_norm] [height_norm].
- (3) Normalizes the values by the image's width and height to be within a [0, 1] range.
- (4) Saves the output as a .txt file for each image, with each line representing one detected object.

3.3 Modeling Approaches

To identify the optimal model for piece detection, we explored two main deep learning architectures as recommended by the project proposal: YOLO and Faster R-CNN.

Our primary approach involved fine-tuning several pretrained YOLO models, experimenting with various model sizes to balance performance and computational cost: YOLOv11n (nano), YOLOv11s

(small), YOLOv11m (medium), YOLOv11l (large), and YOLOv11x (extra-large).

The training configurations were:

- All models were initialized with pretrained weights.
- Input images were resized to 1856×1856 pixels to preserve detail.
- Most models were trained for 20 epochs.
- To maximize accuracy, the YOLOv11m model was trained for an extended period of 50 epochs.
- Batch sizes were adapted based on GPU memory constraints, ranging from 5 for the smallest model (n) to 1 for the largest (x).

3.4 Results

The various modeling strategies were evaluated based on their Mean Average Precision (mAP50, mAP50-95) on the validation and test sets. All YOLO variants demonstrated high performance on the test set but in order to increase the performance in the task 3.2 we trained again the yolo11m on 50 epochs. Looking at the results from yolo there is a big probability of yolo being pretrained on this dataset, it might justify the performance.

The final results for the best model are summarized below:

Table 2: Mean Average Precision (mAP50-95) for Different Data Splits

Data Set	Precision	Recall	(mAP50-95)
Yolo11x	0.998	0.998	0.882
Yolo11l	0.999	0.999	0.877
Yolo11m	1	0.999	0.879
Yolo11s	1	0.999	0.885
Yolo11n	0.999	0.998	0.875

3.5 Conclusion

For the task of detecting chess pieces, fine-tuning pretrained YOLO models proved to be a highly effective and successful strategy. The combination of a powerful architecture, high-resolution images, and sufficient training allowed the models to learn the distinct features of each piece with remarkable accuracy. It's ability to generate precise bounding boxes and correct classifications provides a solid and reliable foundation for the next stage of the project: creating the board's digital twin.

4 TASK 3.2

4.1 Methodology

We applied 3 different approaches in :

- As described in the paper of the chessRED dataset [1] we are going to use a pretrained model (resnet50, resnet101XT, resnet152) change the output layer to output with 64 outputs that will represent the 64 squares of the board. We created a couple of different version of this problem with multi-labeling 64*13(13 one for each piece + 1 empty) output layer, and with sigmoid activation.

- Using the yolo11m trained on piece detection and calculate the IOU of the bounding boxes of the pieces, with the squares detected in the task1.
- Using the yolo11m trained on the piece detection with another yolo11m pose detection trained on the points of the corners of the board to increase the capabilities to detect the board well enough, after using this technique we noticed that most of the boards were almost perfect, so to further increase performance we stopped using the square detection from the task1 and we just split the wrapped board in a matrix of equal sized squares which gave us the best performance, 95.5% total board accuracy Figure 1 .

4.2 Modeling Approaches

Data augmentation was used in all the different approaches, which increased the performance in all the categories. In our github repository there are 2 folders for the exercise 3.2 3_2simple which is the approach with the two yolo models, and in the 3_2 folder there are 5 to 10 different notebooks that show different resnet models trained using early-stop with plus 100 epochs, unfortunately this approach only had a 8% board accuracy. During development calculating the IOU of each yolo bounding box with the squares turned out to be really slow, so we calculate the distance between the middle point of the bottom edge of the bounding box of the piece with the middle point of the square. To solve some possible conflict where there are multiple pieces with the same square with the smallest distance, the closest piece stays in the square and the other tries to find the next closest, until a distance threshold is reached and the piece doesn't stay in any square and is discarded.

4.3 Results

To better evaluate the results in a more human way we printed in the pipeline the ground Truth Figure 3 , the warped board Figure 4 and the predicted board Figure 5.

- The best Big model was the same as the one from the chessRed dataset [1] we used ResnetXT101_64_X4D with batch size 4 and cross validation loss function result Figure 2.
- Yolo 11m trained on the piece detection with the board and square detection of task1 Figure6 .
- Yolo 11m trained on the piece detection with 50 epochs batch size 2 resolution of 1850px we get this performance with a yolo 11m pose detection to detect the edges of the board trained on 200 epochs 1 .

4.4 Conclusion

In the end the best approach was a combination of state of the art YOLO models, replacing the board/square detection with the board detection was the best one by far with a 95% accuracy on the big testing dataset Figure 1.

The implementation of the models and experiments is publicly available for review¹.

ANEXES

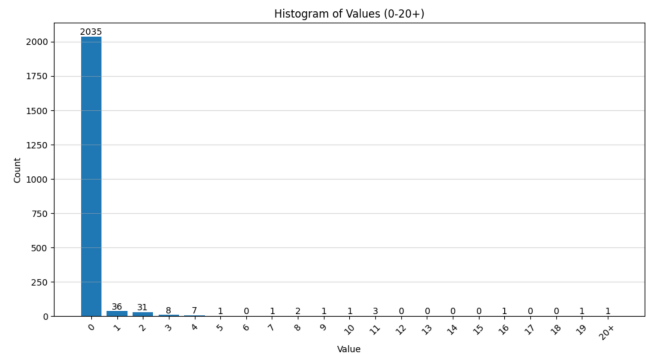


Figure 1: Piece detection with yolo11m pose detection (the number of errors in the boards)

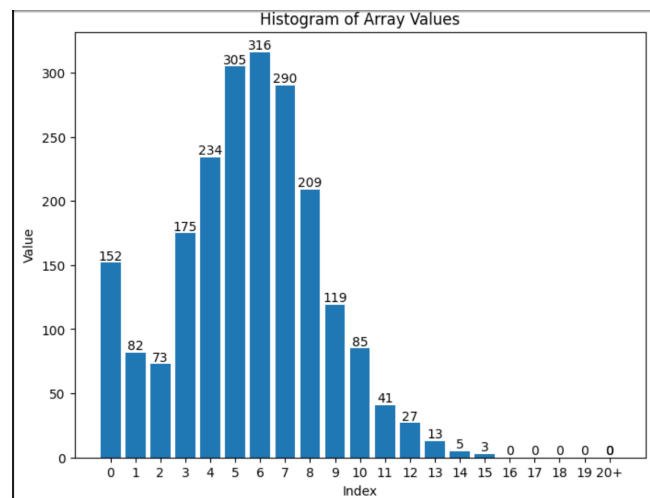


Figure 2: Resnetxt101_64_x4D applied to 64 outputs

¹<https://github.com/Monsterdied/computer-vision/tree/newMain>



Figure 3: Ground Truth

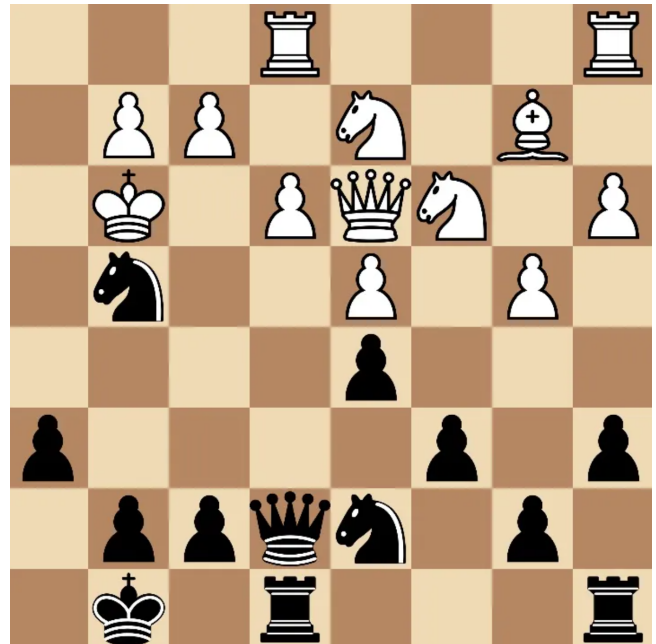


Figure 5: Predicted Board

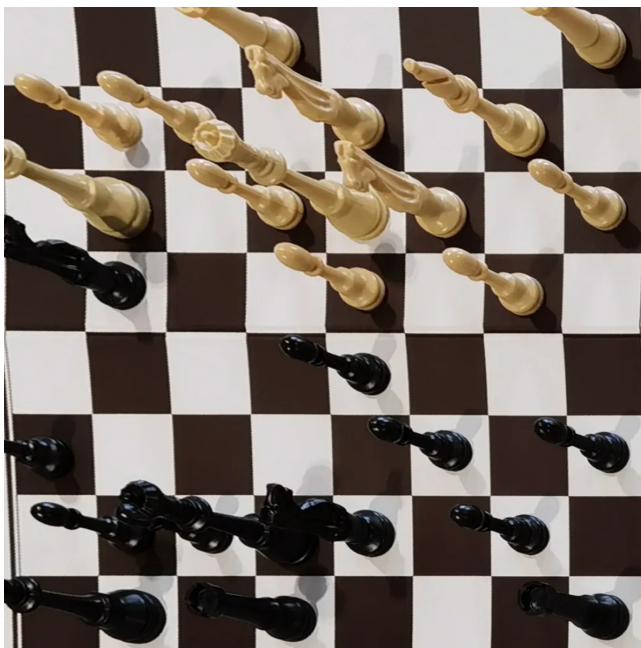


Figure 4: Warped Board

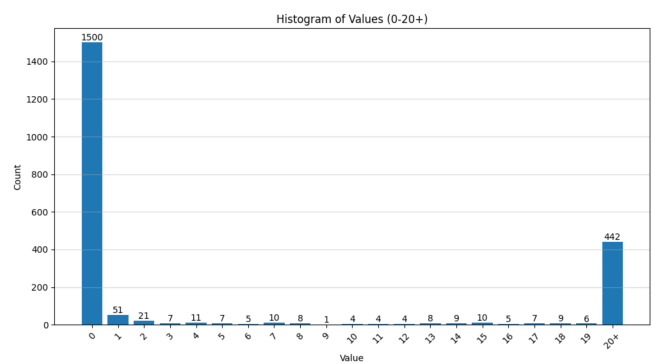


Figure 6: Yolo 11m with task 1 procedures

REFERENCES

- [1] Athanasios Masouris. 2023. *Chess Recognition Dataset (ChessReD)*. <https://doi.org/10.4121/99B5C721-280B-450B-B058-B2900B69A90F.V2>